

1.1 Background and Motivation

Large language models (LLMs) have become central to current work in natural language processing because a single pre-trained model can be adapted to a wide range of downstream tasks. Pre-training provides broad linguistic and contextual knowledge, but it does not guarantee that a model will respond in a way that matches a user's instruction, output expectation, or task framing. Instruction fine-tuning addresses this gap by adapting a pre-trained model with instruction-response pairs rather than with a generic language-modelling objective alone.

This thesis examines one part of instruction fine-tuning that is usually treated as a routine implementation choice: the construction and ordering of mini-batches during training. In standard supervised fine-tuning pipelines, mini-batches are shuffled randomly. That choice is often reasonable, but it also leaves open a practical question. If the model is exposed to different local data structure during training, does the optimization trajectory change in a useful way?

The study focuses on two structured alternatives to fully random batching. The first is semantic grouping, where semantically related instructions are placed into the same mini-batch. The second is curriculum scheduling, where the training order is changed across phases or according to a length-based heuristic. The aim is not to introduce a new model architecture or a new fine-tuning method. The aim is to test whether a change in sample organization can alter instruction fine-tuning behaviour under controlled conditions.



Figure 1.1. Research motivation showing how this thesis narrows from instruction fine-tuning to the unexplored role of mini-batch scheduling.

1.2 Problem Statement

Instruction fine-tuning is commonly performed with randomly constructed mini-batches, yet the effect of batch composition and batch ordering on training dynamics is not well established. In particular, it is unclear whether semantically coherent batches or curriculum-style schedules can improve optimization behaviour, generalization, or generation quality relative to standard random batching.

This thesis addresses the following problem:

How does batch scheduling affect instruction fine-tuning of a large language model under a controlled LoRA-based training setup?

The problem is examined through direct comparison between random batching, semantic grouped batching, two-phase semantic curricula, and length-based curricula.

1.3 Research Objectives

The main objective of the thesis is to evaluate whether batch scheduling strategy affects instruction fine-tuning outcomes.

The specific objectives are:

1. To establish Random batching as a controlled baseline for instruction fine-tuning.
2. To implement semantic grouped batching using sentence embeddings and nearest-neighbour retrieval.
3. To compare two-phase semantic curricula in both directions: Grouped -> Random and Random -> Grouped.
4. To evaluate length-based curricula using Easy -> Hard and Hard -> Easy ordering.
5. To study these strategies across Dolly subsets of 1k, 3k, and 5k examples.
6. To compare methods using optimization metrics and generation metrics.
7. To assess whether observed differences are meaningful relative to seed variation.

1.4 Research Questions

The thesis is guided by the following questions:

1. Does semantic grouped batching improve instruction fine-tuning performance relative to Random batching?
2. Does the ordering of semantic structure across training phases affect learning behaviour?
3. Does Random remain a strong baseline when compared with more structured batching strategies?
4. Do length-based curricula provide a clearer signal than semantic grouping alone?
5. Are any observed gains large enough to be interpreted as practically meaningful under the tested setup?

1.5 Scope of the Study

The study is limited to supervised instruction fine-tuning of google/flan-t5-small with LoRA. The data source is the Databricks Dolly instruction-following dataset, evaluated through 1k, 3k, and 5k subsets. The final production strategies are:

- Random
- Grouped
- Grouped -> Random
- Random -> Grouped
- Easy -> Hard Length
- Hard -> Easy Length

The longer-training hard-to-easy run (exp_032) is analysed separately because it extends the training budget and is not a same-budget comparison with the standard 5k curriculum runs.

The thesis does not cover RLHF, inference-time retrieval augmentation, full-model pre-training, large-scale deployment, or human preference evaluation. It also does not treat mixed batching as part of the final controlled comparison; mixed batching appears only in exploratory material and is not part of the final production experiment matrix.

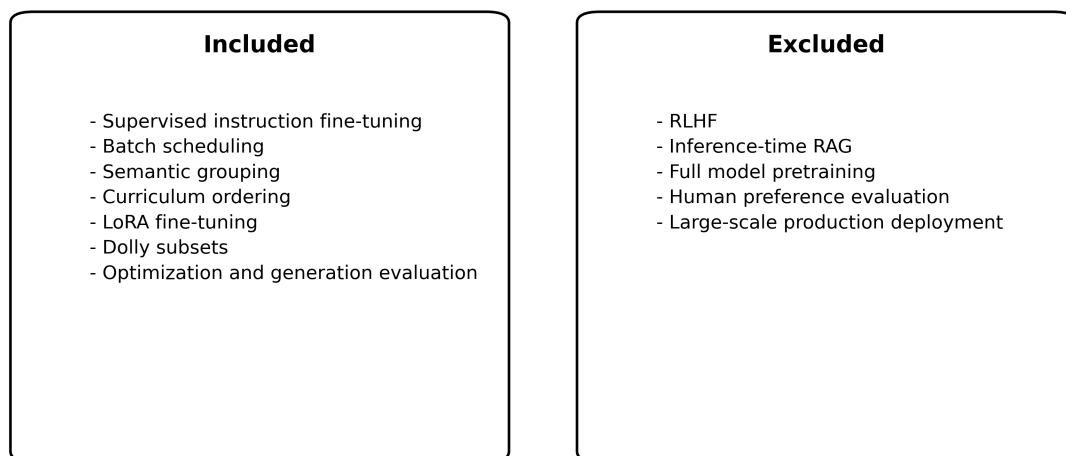


Figure 1.2. Scope of the thesis, separating the investigated components from areas outside the study.

1.6 Significance of the Study

The study is relevant for three reasons. First, it examines a design choice that is present in every fine-tuning pipeline but is rarely isolated as an experimental variable. Second, it provides negative as well as positive evidence. If semantic grouping does not improve

performance, that is still useful because it sets a realistic boundary on what batch-level semantic structure can achieve. Third, it tests whether curriculum-style ordering is more informative than static grouping, which has practical value for future work on efficient fine-tuning.

The contribution of the thesis is therefore not the proposal of a new training algorithm. It is a controlled empirical account of how batch scheduling behaves in instruction fine-tuning under modest resources, with explicit attention to weak effects, seed variation, and implementation caveats.

1.7 Thesis Structure

The remainder of the thesis is organised as follows.

- Chapter 2 reviews the literature on instruction fine-tuning, LoRA, mini-batch training, semantic similarity, curriculum learning, and evaluation metrics.
- Chapter 3 presents the conceptual methodology, including the strategy families, semantic grouping approach, curriculum designs, and evaluation framework.
- Chapter 4 describes the implementation-specific experimental setup, including datasets, preprocessing, model configuration, training settings, samplers, and experiment groups.
- Chapter 5 reports and analyses the results, with separate treatment of optimization outcomes, curriculum effects, generation metrics, and seed variance.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the literature most directly related to the thesis: instruction fine-tuning, parameter-efficient adaptation, mini-batch training, semantic similarity, curriculum learning, and automatic evaluation for generated text. The purpose is not to restate the full history of these topics. It is to identify the specific research gap addressed here: the role of mini-batch construction and scheduling during instruction fine-tuning.

Instruction fine-tuning has been studied extensively at the levels of model design, data curation, alignment, and scale. By contrast, mini-batch scheduling is often treated as a background implementation detail. That asymmetry matters. If batching strategy changes the local gradients seen by the model, then it may influence convergence, generalization, or generation quality without changing the dataset or the architecture.

2.2 Large Language Models and Instruction Fine-Tuning

Modern LLMs are built largely on Transformer-based architectures, which replaced recurrence with self-attention and made large-scale sequence modelling practical [1]. The T5 family extended this line of work by casting many tasks into a unified text-to-text framework, an especially natural fit for instruction-response learning [2]. Flan-T5 further demonstrated that instruction tuning across diverse tasks can improve generalisation and instruction-following behaviour [3]. Related instruction-following work, including InstructGPT, likewise showed that targeted adaptation can materially change how models respond to user instructions [4].

This line of work provides the immediate foundation for the present study. The thesis uses google/flan-t5-small precisely because it is already aligned with the instruction-tuning setting, computationally manageable, and suitable for controlled comparisons. The question is not whether instruction fine-tuning works in general. That is well established. The question is whether training outcomes change when the organization of mini-batches changes.

2.3 Parameter-Efficient Fine-Tuning

Repeated full-parameter fine-tuning is expensive, particularly when a study compares multiple strategies, dataset scales, and seeds. Parameter-efficient fine-tuning addresses that problem by adapting only a small subset of parameters or by introducing lightweight trainable modules while keeping most of the pre-trained model fixed.

LoRA is one of the most widely used methods in this category [5]. By introducing trainable low-rank matrices into selected layers, it reduces memory and training cost while retaining a strong basis for controlled comparison. In the present thesis, LoRA is not itself the object of investigation. It is the practical mechanism that makes repeated multi-seed comparison feasible.

2.4 Mini-Batch Training as an Experimental Variable

Mini-batch training is standard practice in neural network optimisation. Instead of computing a gradient over the full dataset, training proceeds with smaller subsets of examples, which provide stochastic gradient estimates and make training computationally feasible.

In most supervised fine-tuning pipelines, mini-batches are formed randomly. Random batching avoids strong ordering bias, mixes heterogeneous samples, and is often a sensible default. In instruction-tuning datasets, that diversity may be particularly useful because batches can contain different task types, response styles, and contextual structures.

What is less established is whether mini-batch construction should remain entirely random once semantic structure is available. A semantically coherent batch may offer a more locally consistent learning signal. At the same time, that coherence reduces within-batch diversity. The literature gives reasons to consider both possibilities, but it does not provide a clear answer for instruction fine-tuning.

2.5 Semantic Similarity and Retrieval-Based Grouping

Semantic grouping depends on two technical ingredients: sentence-level embeddings and efficient similarity search. Sentence-BERT made semantically meaningful sentence embeddings practical for retrieval and clustering tasks [6], while FAISS provided a scalable way to perform nearest-neighbour search over dense vector collections [7].

Together, these methods make semantic mini-batch construction straightforward in engineering terms. Instruction texts can be embedded, indexed, and grouped by nearest-neighbour structure. What remains uncertain is whether that grouping improves fine-tuning. The existence of a semantic retrieval mechanism does not by itself justify a claim about optimization benefit. That is an empirical question.

2.6 Curriculum Learning and Training Order

Curriculum learning introduced the broader idea that the order of training examples can influence learning [8]. In the classical formulation, models are exposed to easier examples first and harder examples later. Subsequent work has shown that ordering effects can be task-dependent, and in some settings reverse curricula can also be useful.

This literature is relevant because it shifts the focus from static composition to temporal structure. In the present study, that idea is adapted to mini-batch scheduling in two ways. First, semantic structure can be introduced earlier or later through two-phase schedules. Second, example order can be approximated through a length-based heuristic. These designs make it possible to test not only whether structure matters, but when structure might matter.

2.7 Evaluation of Instruction-Tuned Models

Evaluation in instruction fine-tuning usually combines optimization-based and generation-based criteria. Training loss and evaluation loss provide direct information about fit to the observed data, while the generalization gap offers a simple view of how training and evaluation behaviour diverge. In a two-phase setup, the change in evaluation loss during the second phase is also informative because it captures the contribution of the phase transition itself.

Automatic generation metrics add another perspective. ROUGE captures lexical overlap [9], while BERTScore uses contextual embeddings to compare generated and reference text more semantically [10]. Neither metric is a complete substitute for human judgment, but together they provide a practical basis for consistent comparison when manual evaluation is outside scope.

2.8 Position of the Present Study

The present thesis sits at the intersection of several established areas: instruction fine-tuning, parameter-efficient fine-tuning, semantic retrieval, curriculum learning, and automatic evaluation. Its contribution is narrower than any one of those fields. It does not propose a new Transformer architecture, a new PEFT method, or a new evaluation metric. Instead, it isolates batch scheduling as the central research variable.

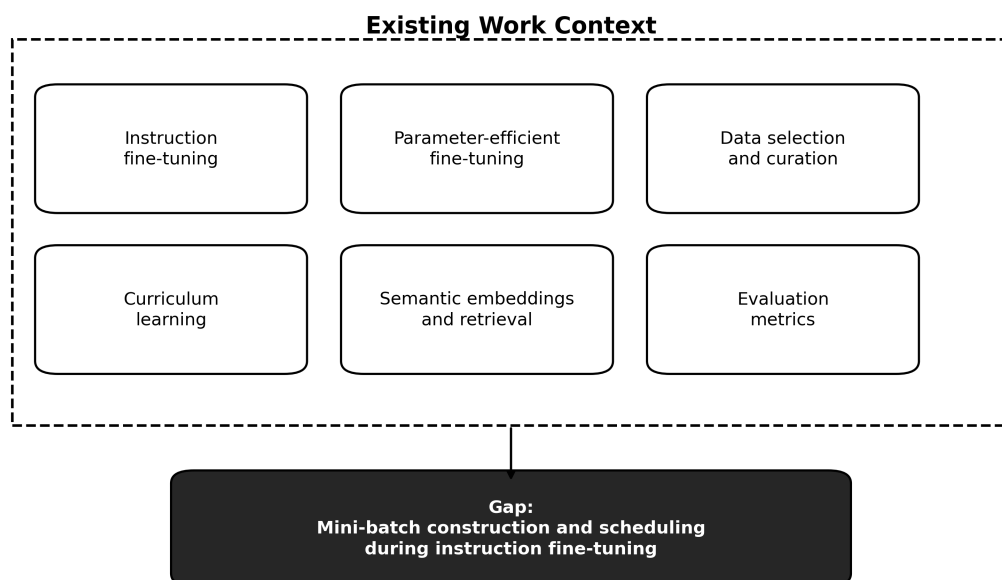


Figure 2.1. Positioning of the present study relative to existing work in instruction fine-tuning, curriculum learning, PEFT, and semantic retrieval.

2.9 Research Gap

The literature supports the plausibility of structured batching, but it does not establish that such structure should help instruction fine-tuning in practice. Existing work has concentrated more on model scale, task coverage, alignment, and data selection than on the composition of individual mini-batches or the timing of structure across training phases.

That gap can be stated precisely: instruction fine-tuning lacks careful empirical comparison of mini-batch construction and scheduling strategies under a controlled, multi-seed setup. This thesis addresses that gap by comparing Random batching with semantic grouping, two-phase semantic curricula, and length-based curricula across multiple dataset scales.

2.10 Chapter Summary

The literature shows why this problem is worth testing. Instruction fine-tuning is well established, LoRA makes repeated experiments feasible, semantic embeddings and FAISS make semantic grouping technically practical, and curriculum learning provides a framework for thinking about training order. What remains unresolved is whether these ingredients improve instruction fine-tuning once they are translated into mini-batch scheduling strategies.

The next chapter sets out the conceptual methodology used to answer that question.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
- [2] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, 2020.
- [3] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, et al., "Scaling Instruction-Finetuned Language Models," *Journal of Machine Learning Research*, 2024.
- [4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., "Training Language Models to Follow Instructions with Human Feedback," *Advances in Neural Information Processing Systems*, 2022.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022.
- [6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of EMNLP*, 2019.
- [7] J. Johnson, M. Douze, and H. Jegou, "Billion-scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, 2019.
- [8] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," *International Conference on Machine Learning*, 2009.
- [9] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," *Text Summarization Branches Out*, *ACL*, 2004.

[10] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," International Conference on Learning Representations, 2020.

Chapter 3: Methodology

3.1 Research Design

The study is a controlled comparative experiment on instruction fine-tuning. The independent variable is the batch scheduling strategy. The base model family, fine-tuning approach, dataset source, and evaluation protocol are held constant within each comparable experiment block. This design makes it possible to attribute observed differences, however small, to changes in batch construction or ordering rather than to changes in architecture or data source.

The methodological emphasis is empirical rather than algorithmic. The thesis does not propose a new model, a new embedding method, or a new optimizer. It asks whether the organisation of training samples changes the behaviour of an otherwise fixed instruction fine-tuning pipeline.



Figure 3.1. Overall methodology workflow from dataset preparation through batch scheduling, LoRA fine-tuning, multi-seed evaluation, and result analysis.

3.2 Strategy Families

The final production comparison includes three strategy families.

1. Static strategies: Random and Grouped.
2. Two-phase semantic curricula: Grouped -> Random and Random -> Grouped.
3. Length-based curricula: Easy -> Hard Length and Hard -> Easy Length.

Mixed batching appears in exploratory notebook material, but it is not part of the final controlled production comparison and is not treated as a main thesis result.

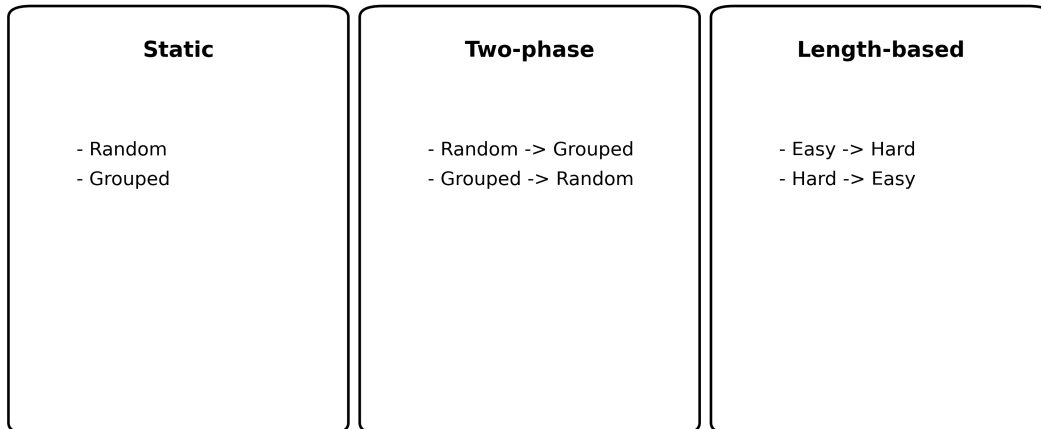


Figure 3.2. Production strategy families evaluated in the thesis: static batching, two-phase curricula, and length-based curricula.

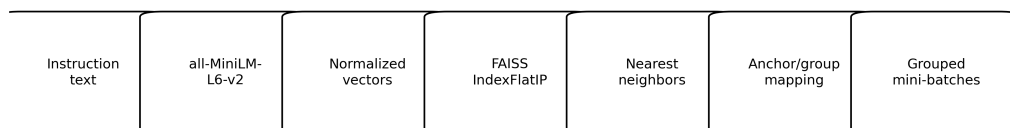
3.3 Semantic Mini-Batch Construction

Semantic grouping is based on a simple idea: instructions that are close in embedding space may provide a more locally coherent training signal when placed into the same mini-batch. To test that idea, the methodology treats semantic grouping as a reproducible pipeline rather than as an informal heuristic.

The conceptual steps are:

1. Convert instruction text into dense sentence embeddings.
2. Normalize the embeddings for similarity search.
3. Build a nearest-neighbour structure over the embedding set.
4. Select anchor examples and retrieve nearby neighbours.
5. Form grouped mini-batches from the anchor-neighbour sets.

This design increases within-batch semantic coherence, but it also reduces within-batch diversity. The thesis therefore treats semantic grouping as a hypothesis to be tested rather than as an assumed improvement over Random batching.



Notes: top_k = 32 for 1k, top_k = 8 for 3k/5k, batch size = 8

Figure 3.3. Schematic of semantic mini-batch construction using sentence embeddings, FAISS nearest-neighbor retrieval, and anchor-based grouping.

3.4 Curriculum Scheduling Designs

The study distinguishes between composition and ordering. Static grouping changes which examples appear together. Curriculum scheduling changes when a particular type of batch structure is presented during training.

Two semantic curriculum directions are evaluated:

- Random -> Grouped: the model first sees diverse batches, then more semantically focused batches.
- Grouped -> Random: the model first sees semantically structured batches, then later returns to random mixing.

Two length-based curricula are also evaluated:

- Easy -> Hard Length: shorter examples first, longer examples later.
- Hard -> Easy Length: longer examples first, shorter examples later.

These curricula are designed to test whether training order influences the optimization path, even when the model and data source remain fixed.

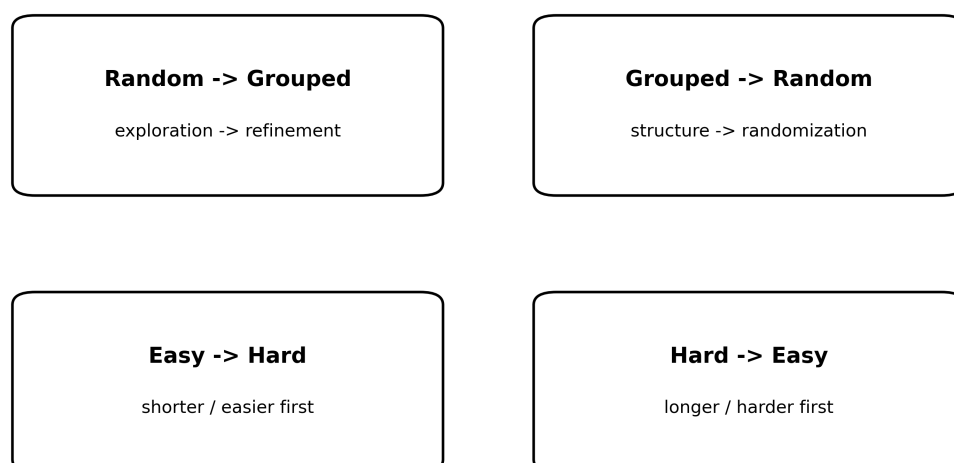


Figure 3.4. Curriculum scheduling designs evaluated in the study, spanning semantic ordering and length-based ordering.

3.5 Experimental Staging

The methodological workflow progresses in stages rather than in a single monolithic comparison.

1. Early controlled 1k experiments establish that the scheduling implementations behave as intended.
2. Longer 1k runs test whether short-run observations persist under a larger training budget.
3. The 3k and 5k blocks test whether any pattern survives scaling.
4. The final 5k length-curriculum runs examine whether a difficulty-style ordering signal is more promising than semantic grouping alone.

This staged design matters because a method that appears promising in a short pilot may weaken or reverse when the data scale or training duration changes.

3.6 Multi-Seed Protocol

Key experiments are repeated with seeds 13, 21, and 42. The purpose of the multi-seed design is not to claim statistical significance from a small sample, but to reduce the risk of treating a single training run as representative. Means and standard deviations are therefore reported alongside endpoint metrics.

The study treats seed variation as part of the substantive interpretation. Where method differences are of similar scale to seed variation, the results are read cautiously.

3.7 Evaluation Framework

The evaluation framework combines optimization-based and generation-based criteria.

Optimization metrics:

- final train loss
- final eval loss
- generalization gap
- phase-wise delta eval during Phase 2

Generation metrics:

- ROUGE-1
- ROUGE-2
- ROUGE-L
- ROUGE-Lsum

- BERTScore F1

This combination is necessary because endpoint loss alone does not fully describe instruction-following behaviour, while automatic generation metrics alone do not explain the training dynamics that produced a result.

3.8 Validity and Reliability Considerations

Several choices are intended to improve reliability.

- Random batching is treated as a serious baseline rather than as a weak default.
- Comparable blocks keep the model family, fine-tuning method, and reporting pipeline fixed.
- Multi-seed aggregation reduces the weight of any single run.
- Multiple dataset scales reduce the chance that a conclusion depends entirely on a pilot setting.
- Both optimization and generation metrics are reported, with caveats where the generation aggregates conflict with some per-seed JSON files.

The methodology therefore supports modest claims well. It is not designed to justify sweeping claims from very small numerical gaps.

3.9 Chapter Summary

This chapter has defined the conceptual methodology of the study: a controlled comparison of batch scheduling strategies for instruction fine-tuning, centred on semantic grouping, curriculum ordering, and length-based scheduling. The next chapter moves from this conceptual design to the implementation-specific setup used in the actual experiment runs.

Chapter 4: Experimental Setup

4.1 Experimental Pipeline

This chapter describes the implemented setup used to execute the experiments reported later in Chapter 5. The conceptual design has already been introduced in Chapter 3. The focus here is operational: dataset variants, preprocessing, model configuration, sampler implementation, experiment groups, evaluation procedures, and known setup caveats.

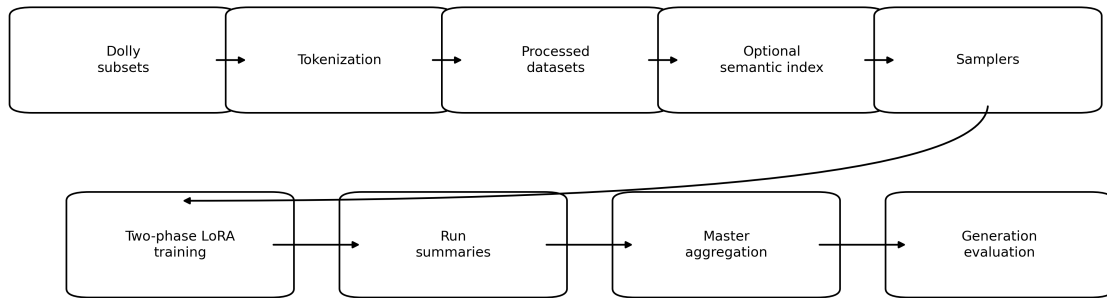


Figure 4.1. Implementation-specific experimental pipeline from Dolly subset preparation through tokenization, sampling, training, aggregation, and generation evaluation.

4.2 Dataset Variants and Preprocessing

The experiments use subsets of the Databricks Dolly instruction-following dataset. Three scales are used in the final study.

Table 4.1. Dataset scales used in the experiments.

Dataset Variant	Train Size	Evaluation Size	Total Size	Main Use
Dolly small 1k	900	100	1,000	Pilot and controlled 1k experiments
Dolly 3k	2,700	300	3,000	Intermediate-scale comparison
Dolly 5k	4,500	500	5,000	Main validation and curriculum experiments

For the 3k and 5k variants, the raw Dolly source was deterministically shuffled with seed 42 and then split 90/10 into train and evaluation partitions. For the 1k runs, the executed split was also 900/100, but the raw-subset provenance is weaker than for the 3k and 5k variants and should be acknowledged as a reproducibility caveat.

Table 4.2. Preprocessing and tokenization settings.

Setting	Dolly 1k	Dolly 3k / 5k
Input formatting	Instruction template with optional context block	Instruction only, or instruction plus context
Target text	Response	Response
Max input / label length	256	512
Truncation	True	True
Padding	False in the recovered 1k notebook path	padding="max_length"
Retained processed columns	Not fully persisted in a canonical config file	input_ids, attention_mask, labels, raw_idx

4.3 Base Model and LoRA Configuration

All experiments use google/flan-t5-small as the base model. The final production scripts load the model through a sequence-to-sequence interface, while the local model configuration identifies the architecture as T5ForConditionalGeneration.

Table 4.3. Base model setup.

Component	Value
Base model	google/flan-t5-small
Local model path	models/flan-t5-small
Architecture	T5ForConditionalGeneration
Model type	Encoder-decoder
Tokenizer family	T5 tokenizer
Maximum model length	512

Fine-tuning is performed with LoRA in all production experiments.

Table 4.4. LoRA configuration.

Hyperparameter	Value
Rank r	16
Alpha	32
Dropout	0.05
Target modules	q, v
Task type	SEQ_2_SEQ_LM
Bias	none

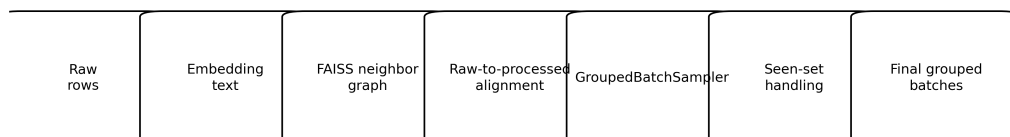


Figure 4.2. Implementation-specific semantic grouping pipeline from raw examples through FAISS retrieval, alignment, sampler logic, and grouped mini-batch construction.

4.4 Training Configuration

The final scripted experiments use a shared Hugging Face training configuration across comparable blocks.

Table 4.5. Core training configuration.

Setting	Value
Optimizer	AdamW Torch
Scheduler	Linear
Warmup steps	0
Per-device train batch size	8
Per-device eval batch size	8
Gradient accumulation steps	4
Effective train batch size	32
Weight decay	0.01
Logging steps	10
Evaluation strategy	Steps
Evaluation steps	50
Save strategy	Epoch
Save total limit	2
FP16	False
BF16	False
predict_with_generate during training	False

Two-phase training is used throughout the controlled production comparisons. Phase 1 uses a learning rate of 7e-05, and Phase 2 uses 5e-05.

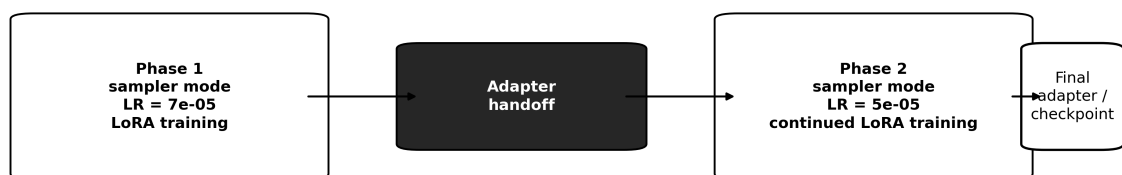


Figure 4.3. Two-phase training flow showing phase-specific sampler modes, learning rates, adapter handoff, and the final checkpoint output.

Table 4.6. Training duration by experiment block.

Experiment Block	Epochs per Phase
exp_012-exp_015	0.2
exp_018-exp_031	0.5

Experiment Block	Epochs per Phase
exp_032	1.0

4.5 Seed Handling and Reproducibility Caveat

The final experiment groups use three seeds: 13, 21, and 42. These values are passed to the custom samplers to control batch ordering. The persisted training arguments, however, also show seed=42, and no explicit global `set_seed`, `torch.manual_seed`, or equivalent call was recovered for the final production runs.

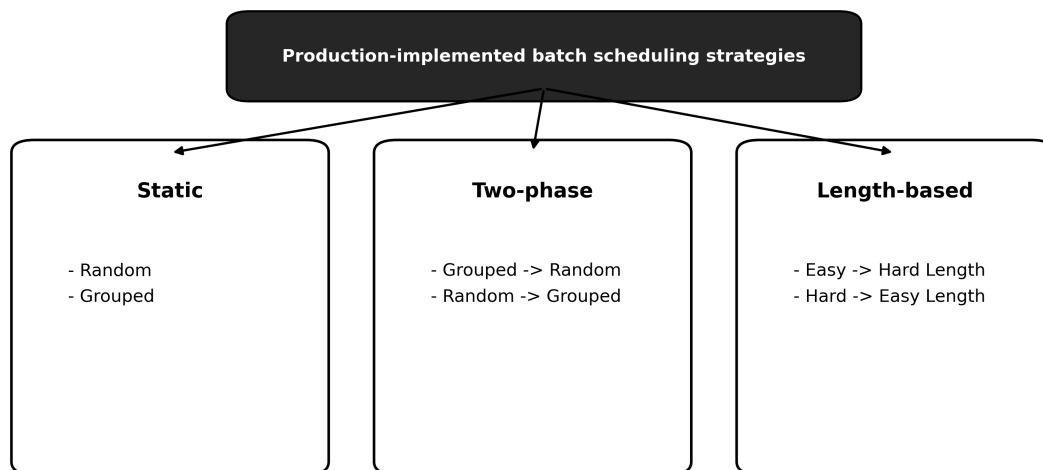
This matters for interpretation. The reported multi-seed variation clearly reflects differences in sampler ordering, but it may not reflect fully independent model-level random initialization in the strongest possible sense. The thesis therefore treats seed-based variability cautiously, especially where endpoint differences are very small.

4.6 Batch Scheduling Implementations

The final controlled comparison uses six production-implemented scheduling strategies:

- Random
- Grouped
- Grouped -> Random
- Random -> Grouped
- Easy -> Hard Length
- Hard -> Easy Length

Mixed batching was explored separately, but no final production mixed sampler or controlled mixed-batching configuration was found in the configs, src, or scripts paths used for the main experiment set.



Note: mixed batching was exploratory only and is not part of the final production comparison.

Figure 4.4. Taxonomy of the production-implemented batch scheduling strategies used in the final experiment matrix.

Mixed batching was exploratory only and is not part of the final production comparison.

4.6.1 Random Batching

Random batching is implemented as the baseline sampler. It shuffles the processed training indices and emits contiguous batches of size 8.

4.6.2 Semantic Grouped Batching

Grouped batching uses an embedding-based nearest-neighbour pipeline. Each processed training row is aligned to a raw row, nearest neighbours are retrieved from the semantic index, and grouped mini-batches are assembled while tracking a global seen set to reduce repeated emission of examples.

Table 4.7. Semantic grouping configuration.

Dataset	Embedding Text	Embedding Model	FAISS Index	Similarity	Top-k	Batch Size
Dolly 1k	Instruction only	all-MiniLM-L6-v2	IndexFlatIP	Cosine via normalized inner product	32	8
Dolly 3k	Instruction + context	all-MiniLM-L6-v2	IndexFlatIP	Cosine via normalized inner product	8	8
Dolly 5k	Instruction + context	all-MiniLM-L6-v2	IndexFlatIP	Cosine via normalized inner product	8	8

The 1k semantic index differs materially from the 3k and 5k indices. That inconsistency is a setup caveat and should be made explicit.

4.6.3 Two-Phase Semantic Curricula

The two-phase semantic curricula reuse the Phase 1 adapter in Phase 2 rather than restarting from the base model.

- Random -> Grouped: random batches in Phase 1, grouped batches in Phase 2.
- Grouped -> Random: grouped batches in Phase 1, random batches in Phase 2.

4.6.4 Length-Based Curricula

Length-based curricula are implemented through a curriculum sampler that orders samples by a length score. In the final implementation, the score is based on `len(input_ids)` plus the count of non-pad label tokens.

This setup has an important caveat. For the 3k and 5k processed datasets, the input side is padded to a fixed length of 512. That means the input-length contribution is effectively constant, so the ordering is driven mainly by the label side rather than by full prompt complexity. The length curriculum should therefore be interpreted as a label-length-oriented heuristic rather than as a direct measure of overall task difficulty.

4.7 Experiment Groups

Table 4.8. Main experiment groups used in the final thesis.

Experiment IDs	Dataset Size	Methods	Training Duration
exp_012-exp_015	1k	Random, Grouped, Grouped -> Random, Random -> Grouped	0.2 epochs per phase
exp_018-exp_021	1k	Random, Grouped, Grouped -> Random, Random -> Grouped	0.5 epochs per phase
exp_022-exp_025	3k	Random, Grouped, Grouped -> Random, Random -> Grouped	0.5 epochs per phase
exp_026-exp_029	5k	Random, Grouped, Grouped -> Random, Random -> Grouped	0.5 epochs per phase
exp_030-exp_031	5k	Easy -> Hard Length, Hard -> Easy Length	0.5 epochs per phase
exp_032	5k	Hard -> Easy Length	1.0 epoch per phase

4.8 Evaluation Setup

The evaluation procedure includes both loss-based and generation-based components.

Loss-based reporting uses the processed evaluation split associated with each dataset variant and aggregates:

- final train loss
- final eval loss
- generalization gap
- Phase 2 delta eval

Generation evaluation is available for the 5k batching and length-curriculum experiments. The evaluation script reconstructs the evaluation split using the raw data and a deterministic `train_test_split(test_size=0.1, seed=42)` procedure.

Table 4.9. Generation evaluation configuration.

Setting	Value
Generation batch size	8
Max input length	512
Max new tokens	128
do_sample	False
ROUGE package	<code>evaluate.load("rouge")</code>
BERTScore package	<code>evaluate.load("bertscore")</code>

4.9 Environment and Data-Reporting Caveats

Several caveats affect interpretation but do not invalidate the experiment set.

1. The 1k subset provenance is less fully documented than the 3k and 5k subsets.
2. The 1k semantic index differs from the 3k and 5k semantic indices in both text mode and effective neighbour width.
3. The seed protocol clearly changes sampler order, but may not fully reflect independent global random-state control.
4. The length curriculum is influenced strongly by label-length variation because the input side is padded to a fixed maximum length.
5. Aggregated generation summary CSVs conflict with some overlapping per-seed JSON files and should therefore be used consistently and cautiously.

4.10 Chapter Summary

This chapter has described the actual experimental configuration used in the thesis: the dataset variants, preprocessing choices, model and LoRA setup, training parameters, sampler implementations, experiment groups, and evaluation procedures. These implementation details provide the basis for the results reported in Chapter 5.

Chapter 5: Results and Analysis

5.1 Overview

This chapter presents the results of the batch scheduling experiments described in Chapter 4. The analysis is organised around three questions. First, how do the compared strategies differ in endpoint optimization performance? Second, do semantic grouping or curriculum ordering provide any clear advantage over Random batching? Third, how large are the observed differences once seed variation and evaluation caveats are taken into account?

Unless stated otherwise, the canonical source for optimization results is reports/master/master_summary_table.csv. Per-seed stability comments are grounded in reports/master/master_per_seed_results.csv. Generation results are taken from the two aggregated generation summary CSVs for the 5k experiments, with the explicit caveat that those aggregates conflict with some overlapping per-seed JSON files.

5.2 Optimization Results by Experiment Block

5.2.1 Dolly 1k Short Training

Table 5.1. Dolly 1k short training comparison.

Method	Experiment ID	N	Seeds	Mean Eval Loss	Std Eval Loss	Mean Train Loss	Mean Generalization Gap	Mean Phase2 Delta Eval
Random	exp_012_random_only_multiseed	3	13,21,42	9.61639912 9231771	0.00012846 433173675 242	38.7701000 63747826	-29.153700 934516056	-0.0013834 635416666 667
Grouped	exp_013_grouped_only_multiseed	3	13,21,42	9.61652692 159017	0.00021044 965469782 815	38.7222095 06564674	-29.105682 584974502	-0.0013230 641682942 708
Grouped -> Random	exp_014_grouped_to_random_multiseed	3	13,21,42	9.61659145 3552246	1.26159253 64802315e-05	38.7713300 9168837	-29.154738 63813612	-0.0013119 379679361 98
Random -> Grouped	exp_015_random_to_grouped_multiseed	3	13,21,42	9.61638673 1465658	9.03947098 320946e-05	38.7225468 95345054	-29.106160 163879395	-0.0013634 363810221 355

Random -> Grouped gives the lowest mean eval loss in this block, while Grouped -> Random gives the highest. The absolute difference between best and worst is 0.000204722086588. This is extremely small. The correct reading is that a possible ordering effect appears in the 1k short setting, but the margin is too narrow to justify a strong claim.

5.2.2 Dolly 1k Longer Training

Table 5.2. Dolly 1k longer training comparison.

Method	Experiment ID	N	Seeds	Mean Eval Loss	Std Eval Loss	Mean Train Loss	Mean Generalization Gap	Mean Phase2 Delta Eval
Random	exp_018_random_only_multiseed_long	3	13,21,42	9.611108779907227	0.00018671569539907427	38.760330539279515	-29.149221759372285	-0.004041353861490886
Grouped	exp_019_grouped_only_multiseed_long	3	13,21,42	9.611552556355795	0.00010812468756713155	38.722173394097226	-29.11062083774143	-0.0038159688313802085
Grouped -> Random	exp_020_grouped_to_random_multiseed_long	3	13,21,42	9.611759503682455	0.00021096115141597821	38.76272142198351	-29.150961918301054	-0.0037781397501627603
Random -> Grouped	exp_021_random_to_grouped_multiseed_long	3	13,21,42	9.611469268798828	0.0002447879672933447	38.722423977322045	-29.11095470852322	-0.003803253173828125

In the longer 1k setting, Random gives the lowest mean eval loss. Grouped -> Random again performs worst. Random -> Grouped remains better than Grouped -> Random, but it does not beat the Random baseline. This weakens any claim that the Random -> Grouped ordering is uniformly favourable.

5.2.3 Dolly 3k

Table 5.3. Dolly 3k comparison.

Method	Experiment ID	N	Seeds	Mean Eval Loss	Std Eval Loss	Mean Train Loss	Mean Generalization Gap	Mean Phase2 Delta Eval
Random	exp_022_random_only_multiseed_3k	3	13,21,42	11.423059145609537	0.0022963785111432176	45.43341644050539	-34.010357294895854	-0.03380934397379557
Grouped	exp_023_grouped_only_multiseed_3k	3	13,21,42	11.425052642822266	0.0006824924120630219	45.453193368837816	-34.02814072601555	-0.032903035481770836
Grouped -> Random	exp_024_grouped_to_random_multiseed_3k	3	13,21,42	11.424866358439127	0.001265824868793103	45.442404103833574	-34.01753774539445	-0.03327242533365885
Random -> Grouped	exp_025_random_to_grouped_multiseed_3k	3	13,21,42	11.425841649373373	0.000684287960140967	45.455506317375246	-34.02966466800187	-0.032347679138183594

At 3k, Random again gives the lowest mean eval loss, and Random -> Grouped becomes the weakest method in the block. The best-to-worst gap is 0.002782503763836. This is still small in absolute terms, but it is large enough to show that the 1k short ordering pattern does not generalise cleanly.

5.2.4 Dolly 5k Main Comparison

Table 5.4. Dolly 5k main comparison.

Method	Experiment ID	N	Seeds	Mean Eval Loss	Std Eval Loss	Mean Train Loss	Mean Generalization Gap	Mean Phase2 Delta Eval
Random	exp_026_random_only_multiseed_5k	3	13,21,42	11.3418038 68611654	0.00403028 783890195 7	45.1223857 6826355	-33.780581 8996519	-0.0782492 955525716 1
Grouped	exp_027_grouped_only_multiseed_5k	3	13,21,42	11.3467791 87520346	0.00250839 086113270 86	45.1715660 63249614	-33.824786 87572927	-0.0753548 940022786 4
Grouped -> Random	exp_028_grouped_to_random_multiseed_5k	3	13,21,42	11.3448311 48783365	0.00099957 140225042 69	45.1361688 7840307	-33.791337 7296197	-0.0772275 924682617 2
Random -> Grouped	exp_029_random_to_grouped_multiseed_5k	3	13,21,42	11.3433799 74365234	0.00289547 505966296 6	45.1589030 5926543	-33.815523 08490019	-0.0758380 889892578 1

The 5k main block is the strongest core comparison because it uses the largest dataset scale among the directly comparable Random/Grouped/curriculum runs. Random again gives the lowest mean eval loss. Grouped performs worst, and the Grouped-versus-Random gap reaches 0.004975318908692. Even here, however, the effect should be described as small rather than decisive.

Figure 5.1 Mean final evaluation loss across experiment blocks

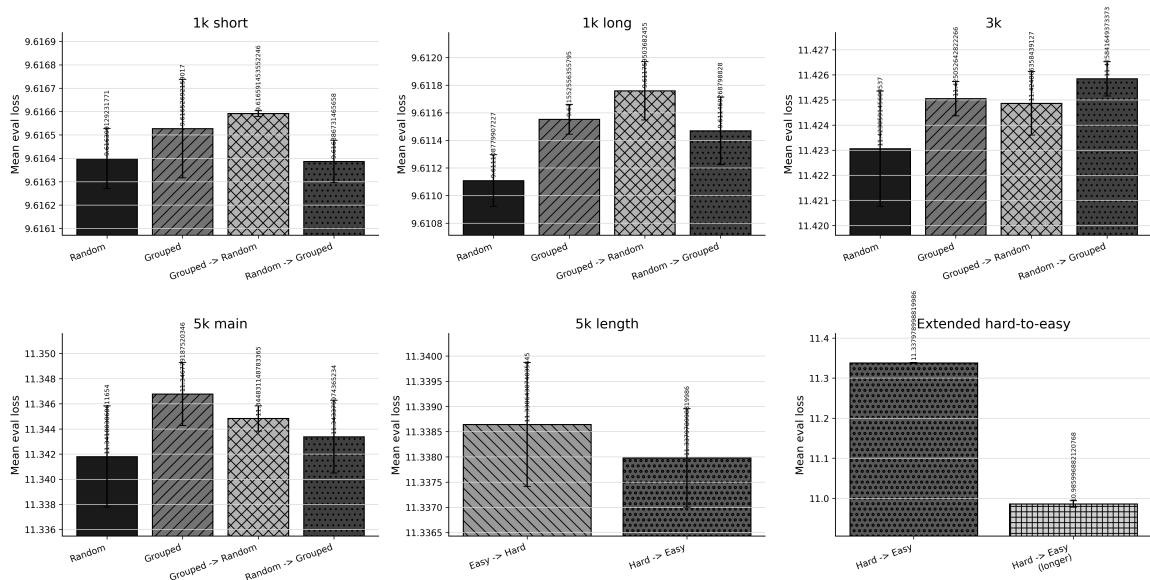


Figure 5.1. Mean final evaluation loss across scheduling strategies and experiment blocks. Lower is better.

5.3 Random vs Semantic Grouping

The central hypothesis behind semantic grouping is that semantically coherent batches may produce more locally consistent gradient updates. The canonical results do not support a clear advantage for this idea.

Table 5.5. Grouped minus Random eval loss across directly comparable blocks.

Block	Grouped Minus Random Eval Loss	Interpretation
1k short	0.000127792358399	Grouped is slightly worse
1k long	0.000443776448568	Grouped is worse
3k	0.001993497212729	Grouped is worse
5k main	0.004975318908692	Grouped is worse

Lower eval loss is better, so all four positive differences indicate that Grouped performs worse than Random in the directly comparable blocks. The pattern is consistent even though the effect size is modest.

This is an important negative result. It shows that increasing within-batch semantic coherence does not automatically improve instruction fine-tuning. A plausible explanation is that Random batching preserves useful task diversity inside each phase, whereas semantic grouping narrows the local gradient signal too strongly for this heterogeneous instruction dataset.

Figure 5.2 Difference from the Random baseline

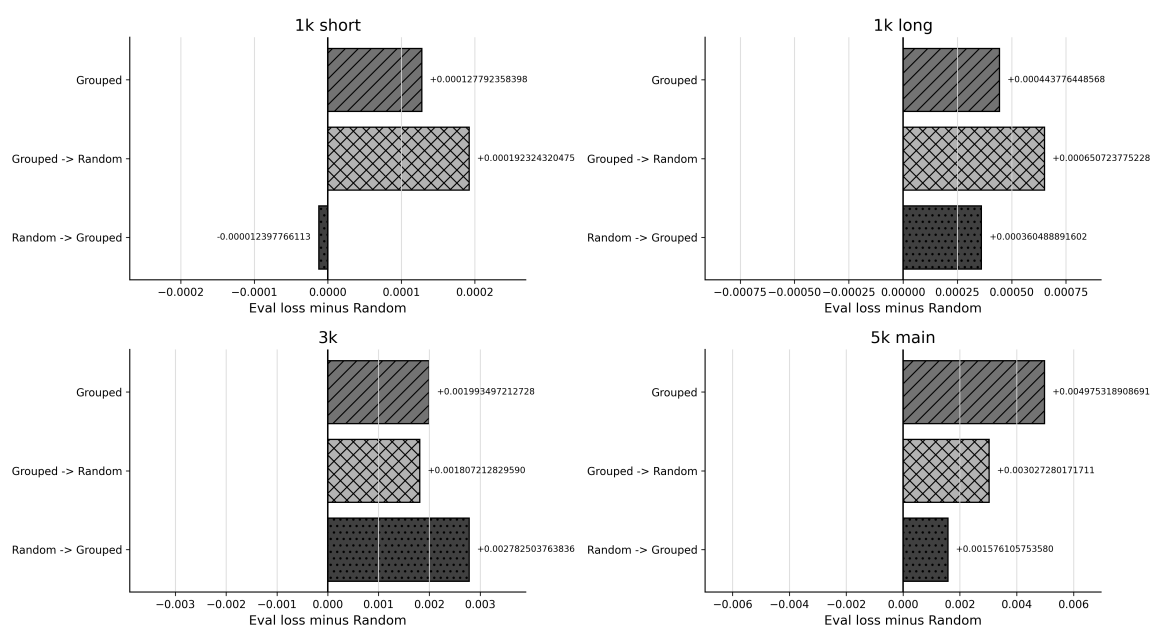


Figure 5.2. Difference in mean evaluation loss relative to the Random baseline. Negative values indicate improvement over Random.

The safest conclusion is therefore straightforward: Random batching remains a strong baseline, and semantic grouped batching does not consistently outperform it.

5.4 Curriculum Ordering and Phase-Wise Behaviour

5.4.1 Curriculum Ordering

The two-phase semantic curricula address a different question from pure grouping. Instead of asking only whether semantic structure helps, they ask whether the timing of semantic structure matters.

Table 5.6. Curriculum ordering comparison.

Block	Random -> Grouped Mean Eval Loss	Grouped -> Random Mean Eval Loss	Better Ordering	Difference
1k short	9.616386731465658	9.616591453552246	Random -> Grouped	-0.000204722086588
1k long	9.611469268798828	9.611759503682455	Random -> Grouped	-0.000290234883627
3k	11.425841649373373	11.424866358439127	Grouped -> Random	0.000975290934246
5k main	11.343379974365234	11.344831148783365	Random -> Grouped	-0.001451174418131

Random -> Grouped performs better in three of the four comparable blocks. The exception is the 3k block, where Grouped -> Random performs better. The margins are narrow throughout. This supports a cautious claim: there is a weak directional tendency in favour of Random -> Grouped, but not a uniform or decisive ordering advantage.

Figure 5.3 Two-phase curriculum ordering comparison

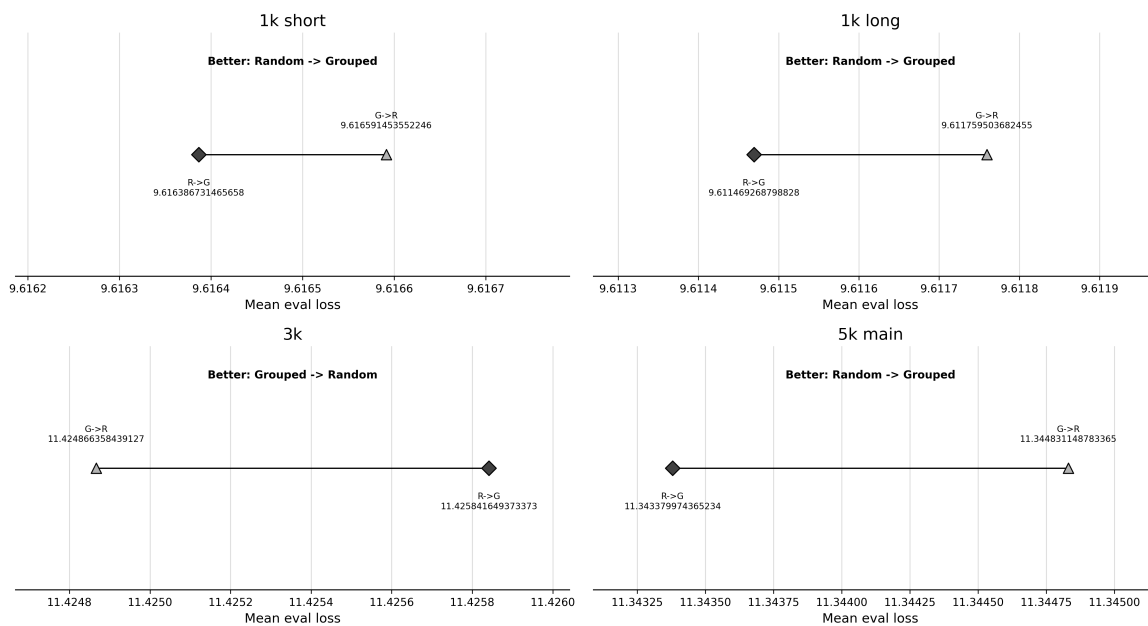


Figure 5.3. Comparison of two-phase semantic curriculum orderings across comparable experiment blocks.

5.4.2 Phase-2 Delta Eval

Phase-2 delta eval measures the change in evaluation loss during the second training phase. Negative values indicate that evaluation loss decreased during Phase 2. More negative values therefore indicate stronger Phase-2 improvement.

Table 5.7. Mean Phase-2 delta eval.

Block	Method	Mean Phase2 Delta Eval
1k short	Random	-0.0013834635416666667
1k short	Grouped	-0.0013230641682942708
1k short	Grouped -> Random	-0.001311937967936198
1k short	Random -> Grouped	-0.0013634363810221355
1k long	Random	-0.004041353861490886
1k long	Grouped	-0.0038159688313802085
1k long	Grouped -> Random	-0.0037781397501627603
1k long	Random -> Grouped	-0.003803253173828125
3k	Random	-0.03380934397379557
3k	Grouped	-0.032903035481770836
3k	Grouped -> Random	-0.03327242533365885
3k	Random -> Grouped	-0.032347679138183594
5k main	Random	-0.07824929555257161
5k main	Grouped	-0.07535489400227864
5k main	Grouped -> Random	-0.07722759246826172
5k main	Random -> Grouped	-0.07583808898925781

All reported Phase-2 delta values in these blocks are negative. Phase 2 therefore improves evaluation loss in every case. Within each same-budget block, however, the differences between methods remain small. Random often has the strongest Phase-2 improvement among the Random/Grouped comparison blocks, which again reinforces the strength of the Random baseline.

Figure 5.4 Phase-2 evaluation-loss change

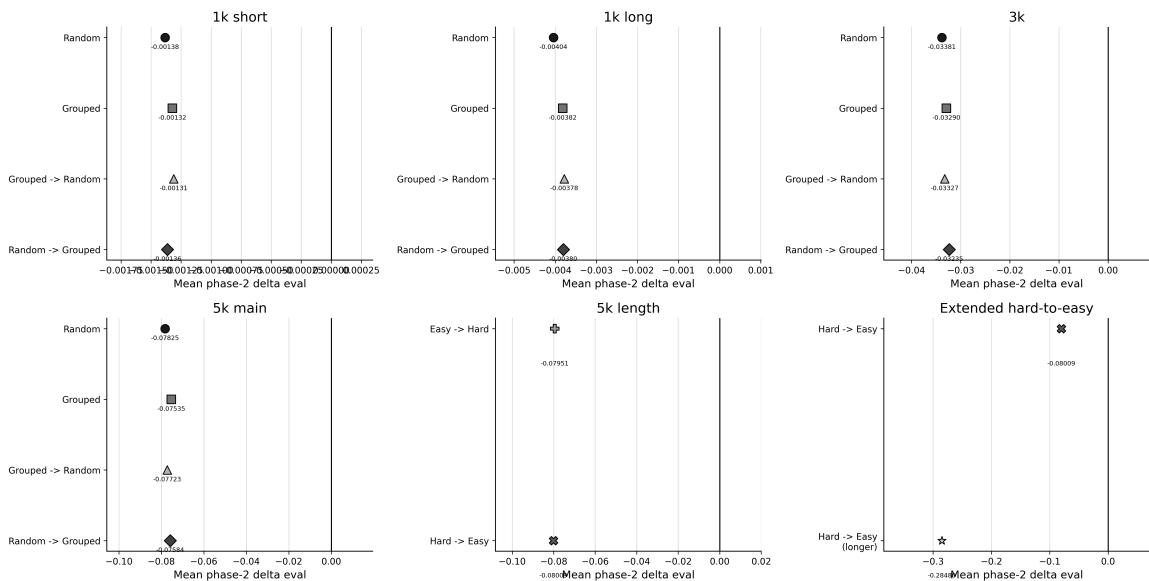


Figure 5.4. Mean evaluation-loss change during Phase 2. More negative values indicate larger phase-2 improvement.

5.5 Length-Based Curriculum Results

5.5.1 Same-Budget Comparison

Table 5.8. Dolly 5k length-curriculum comparison.

Method	Experiment ID	N	Seeds	Mean Eval Loss	Std Eval Loss	Mean Train Loss	Mean Generalization Gap	Mean Phase2 Delta Eval
Easy -> Hard Length	exp_030_easy_to_hard_length_multiseed_5k	3	13,21,42	11.338643074035645	0.0012300206513966253	45.13238579118755	-33.79374271715191	-0.0795135498046875
Hard -> Easy Length	exp_031_hard_to_easy_length_multiseed_5k	3	13,21,42	11.337978998819986	0.0009772446107854716	45.12975311279297	-33.791774113972984	-0.08008988698323567

Hard -> Easy Length gives the lower mean eval loss by 0.000664075215659. It also shows a slightly more negative Phase-2 delta than Easy -> Hard Length. The direction is therefore favourable to Hard -> Easy, but the same-budget margin remains very small.

5.5.2 Longer-Training Extension

Table 5.9. Extended hard-to-easy comparison.

Method	Experiment ID	N	Seeds	Mean Eval Loss	Std Eval Loss	Mean Train Loss	Mean Generalization Gap	Mean Phase2 Delta Eval
Hard -> Easy Length	exp_031_hard_to_easy_length_multiseed_5k	3	13,21,42	11.337978998819986	0.0009772446107854716	45.12975311279297	-33.791774113972984	-0.08008988698323567
Hard -> Easy Length Longer Training	exp_032_hard_to_easy_length_longer_training_5k	3	13,21,42	10.985996882120768	0.008129767881074827	44.02101001784593	-33.03501313572516	-0.2848030726114909

exp_032 gives the lowest mean eval loss reported in the thesis: 10.985996882120768. The absolute difference relative to the standard-budget hard-to-easy run is 0.351982116699218. That result is clearly material, but it must not be treated as a same-budget comparison. The longer run changes both strategy exposure and training duration.

The length-based results also need an implementation caveat. Because the processed inputs are padded to a fixed maximum length, the input-side contribution to the curriculum score is effectively constant in the 3k and 5k processed datasets. The ordering is therefore driven mainly by label-side variation rather than by a full prompt-complexity notion of difficulty.

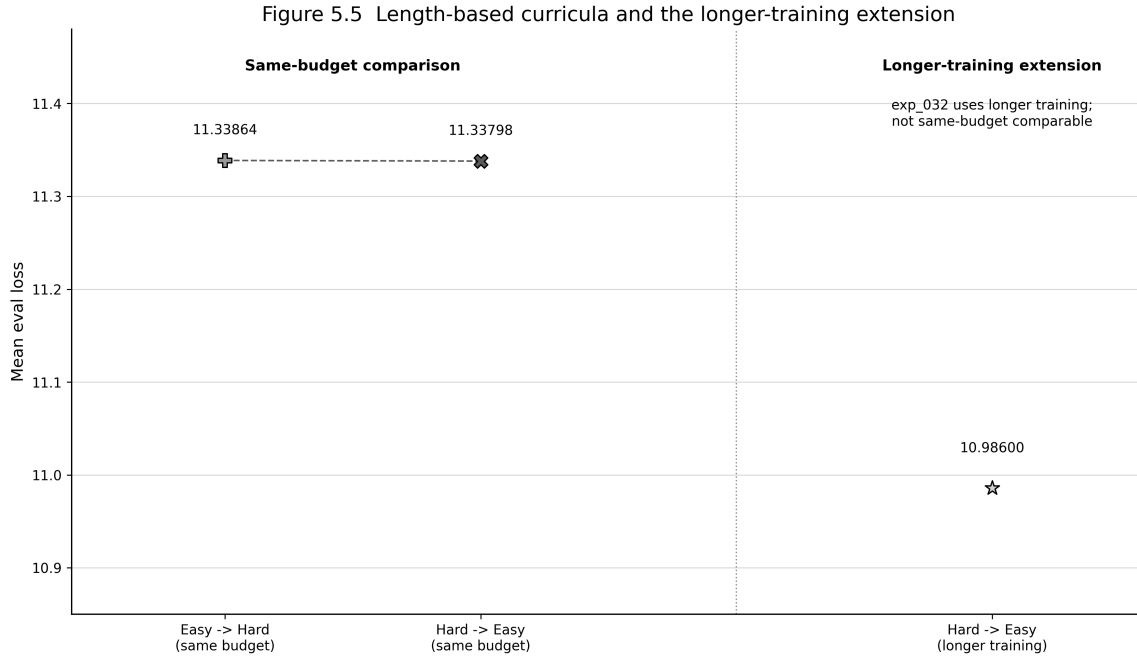


Figure 5.5. Same-budget comparison of length-based curriculum strategies, with the longer-training Hard -> Easy extension shown separately.

The first two points in Figure 5.5 are directly comparable under the same training budget. The separated exp_032 point is shown as a longer-training extension and should not be read as a same-budget comparison.

The most defensible reading is that length-based ordering appears more promising than semantic grouping, Hard -> Easy gives the clearest same-budget curriculum signal, and exp_032 produces the strongest result overall while remaining a longer-training case.

5.6 Generation Quality

Generation quality is reported for the 5k batching and 5k length-curriculum experiments through the aggregated summary CSVs. These aggregates should be used consistently, but they must be read with the explicit caveat that some overlapping per-seed JSON files do not match them exactly.

Table 5.10. Generation metrics on 5k experiments.

Method	Mean ROUGE-1	Mean ROUGE-2	Mean ROUGE-L	Mean ROUGE-Lsum	Mean BERTScore F1
Random	0.1332991221165225	0.05172597342614166	0.12528074807592213	0.1255176984312918	0.8358021107912063
Grouped	0.13515864447757928	0.05267632000153163	0.12741708266264154	0.12762222208271706	0.8365236309369406
Grouped -> Random	0.13573552053686477	0.05290762532282947	0.12741556756984396	0.1279101682552873	0.836249907930692
Random -> Grouped	0.13429778290121783	0.05139917328310489	0.12623164340206786	0.1263183377758188	0.8363362034161885

Method	Mean ROUGE-1	Mean ROUGE-2	Mean ROUGE-L	Mean ROUGE-Lsum	Mean BERTScore F1
Easy -> Hard Length	0.1349535952911857	0.051761622149622204	0.12602883537238033	0.12640203541955244	0.8361860071023305
Hard -> Easy Length	0.13569431685742261	0.052930931439159314	0.12673431381120112	0.12715204647000258	0.8362131182750067

The metric leaders differ by metric:

- best ROUGE-1: Grouped -> Random (0.13573552053686477)
- best ROUGE-2: Hard -> Easy Length (0.052930931439159314)
- best ROUGE-L: Grouped (0.12741708266264154)
- best ROUGE-Lsum: Grouped -> Random (0.1279101682552873)
- best BERTScore F1: Grouped (0.8365236309369406)

The spreads remain narrow:

- ROUGE-1 spread: 0.00243639842034227
- ROUGE-2 spread: 0.001531758156054424
- ROUGE-L spread: 0.00213633458671941
- ROUGE-Lsum spread: 0.0023924698239955
- BERTScore F1 spread: 0.0007215201457343

These values do not support a strong claim that batching strategy substantially changes output quality under the present automated evaluation setup.

Figure 5.6 Generation-quality metrics for 5k experiments

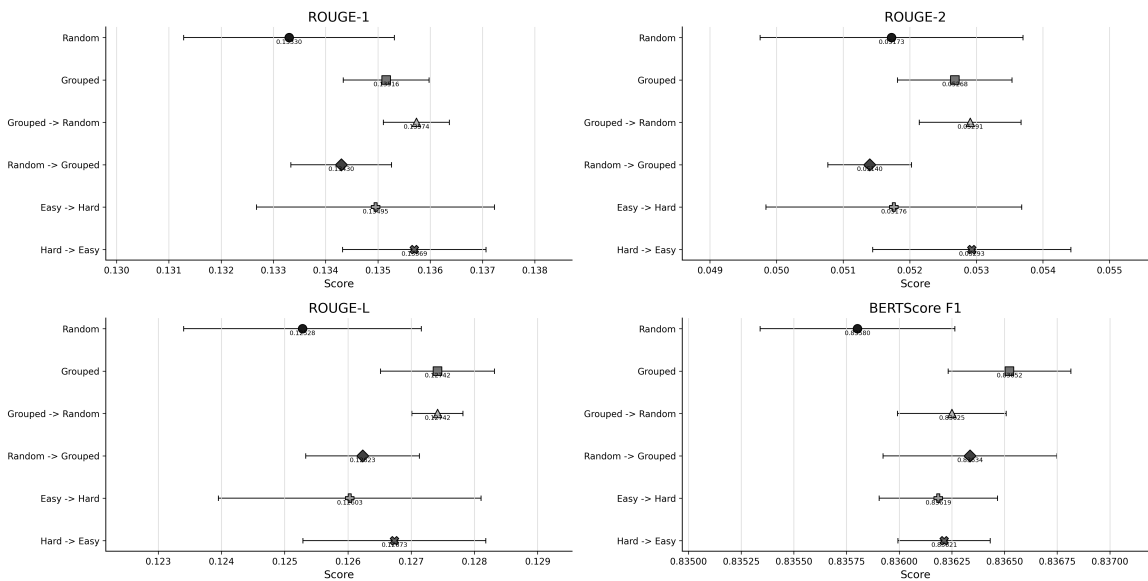


Figure 5.6. Generation-quality metrics for 5k batching and length-curriculum experiments, plotted from the aggregated generation summary CSVs.

Figure 5.6 should be interpreted with the same source caveat stated above: the aggregated generation CSVs are used consistently here even though some overlapping per-seed JSON files do not agree with them exactly.

5.7 Seed Variance and Stability

The interpretation of the endpoint tables depends on the size of the seed variation. In several blocks, the standard deviation is of the same order as the method spread.

Table 5.11. Seed variance and stability summary.

Block	Method	Std Eval Loss	Comment
1k short	Grouped	0.00021044965469782815	Full method spread is smaller than this standard deviation
1k long	Random -> Grouped	0.0002447879672933447	Method differences remain very small in absolute terms
3k	Random	0.0022963785111432176	Method spread is only modestly larger than the Random standard deviation
5k main	Random	0.004030287838901957	Method spread is close to the largest standard deviation in the block
5k length	Easy -> Hard Length	0.0012300206513966253	Same-budget method spread is smaller than both curriculum standard deviations
Extended hard-to-easy	Hard -> Easy Length Longer Training	0.008129767881074827	Larger standard deviation, but still far smaller than the mean gap relative to standard-budget hard-to-easy

This limits the strength of the conclusions. The same-budget differences are often comparable to the observed seed variation. The strongest exception is exp_032, whose mean eval improvement over exp_031 is much larger than its reported standard deviation, but that remains a longer-training rather than same-budget comparison.

Figure 5.7 Seed-level final evaluation loss

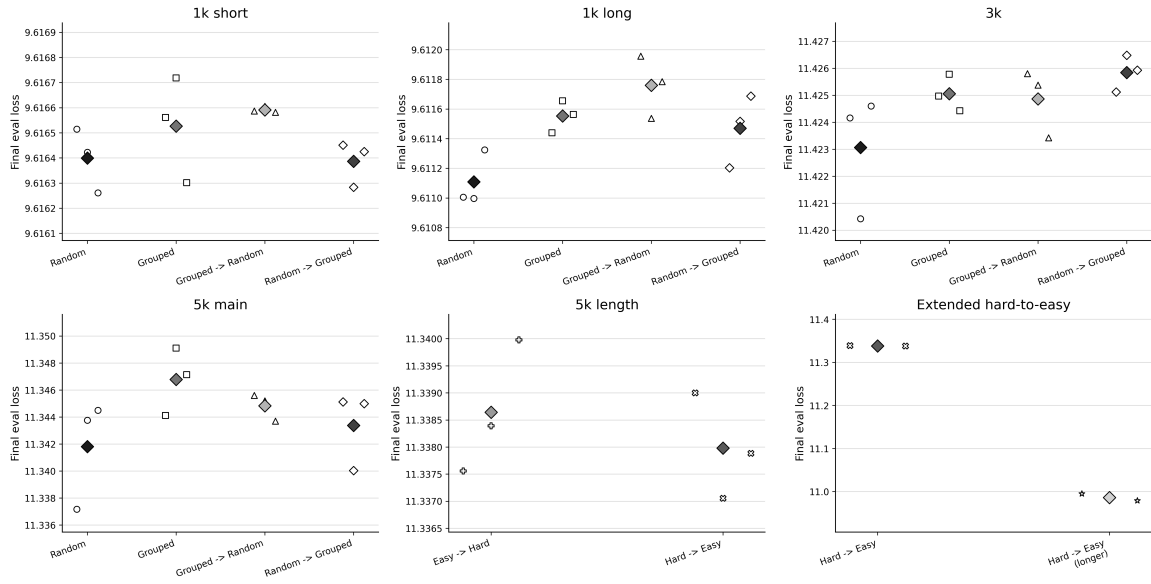


Figure 5.7. Per-seed final evaluation loss showing seed-level variability across scheduling strategies; each experiment includes three seeds.

5.8 Summary of Findings

Five findings can be stated safely.

1. Random batching remains a strong baseline. It gives the best mean eval loss in the 1k longer-training, 3k, and 5k main comparison blocks, and is effectively tied with the best result in the 1k short block.
2. Semantic grouped batching does not consistently outperform Random. In the directly comparable blocks, pure Grouped is worse than Random every time.
3. Curriculum ordering effects are present but small and mixed. Random -> Grouped performs better than Grouped -> Random in three of the four comparable blocks, but not in all of them, and not consistently against the Random baseline.
4. Hard -> Easy Length provides the clearest same-budget curriculum signal. It improves on Easy -> Hard Length at 5k, but only by a very small margin.
5. exp_032 gives the best reported result overall, but it uses longer training and should not be presented as a same-budget comparison.

Taken together, the results suggest that batch scheduling can influence training behaviour, but its effect on final performance is limited under the tested conditions. Random batching remains difficult to beat, semantic grouping alone is not sufficient, and length-based ordering appears more promising than semantic grouping while still requiring conservative interpretation.